

MediByte — Automation QA Assignment (6+ years experience)

- For assessment use only — MediByte is a sample online pharmacy, not a real service. No real orders are placed and no real data is involved.

About the app

MediByte is an online pharmacy storefront. Customers can browse a catalog of over-the-counter (OTC) and prescription (Rx) products, search and filter the catalog, add items to a cart, apply promo codes, check out with a shipping address, and review their past orders and account details.

Over-the-counter (OTC) vs prescription (Rx)

Pharmacies sell two kinds of product, and the distinction may come up as you explore the app:

- **Over-the-counter (OTC)** — everyday medicines and health products anyone can buy without a doctor's authorization: pain relievers, vitamins, antacids, allergy tablets, first-aid supplies, and the like. In a store like this they behave like ordinary retail items — you pick them and buy them.
- **Prescription (Rx)** — medicines that legally require a valid prescription from a licensed prescriber before they can be dispensed. A pharmacy is expected to treat them differently from OTC items: it should establish that a prescription exists, and it handles the associated patient/health information as sensitive data.

Every product in the catalog is labelled as either OTC or Rx.

Your task

Automate the following user journey through the storefront:

1. Log in and go to the account page. Edit the existing saved address (find its "Edit ... address" control) and change the city, then save and confirm the account page shows the new value once out of edit mode.
2. On the same account page, edit the insurance details (find the "Edit" control that belongs specifically to the insurance section, not the address section — the two sit in visually similar cards, each with its own edit and its own destructive control, so this needs more than "find the Edit button"). Change the member ID, save, and confirm the new value is displayed. Note that both sections also validate the *format* of what you enter, not just that it is non-blank — your happy-path values need to be well-formed.
3. Add at least one product to the cart, then go to checkout and place an order, filling in the shipping fields to match the updated account details.
4. Go to the order history page and confirm the order you just placed appears at the top of the list, with a status badge, item count, and total.
5. Open that order's detail page and cross-verify that the shipping address shown there — street, city, region, and postal code — exactly matches what you entered on the account page and at checkout.
6. With more than one order now in the history list, click into a specific order by its order ID and total (not by its position in the list) and confirm the detail page that opens shows that same order's ID, status, items, and total — not a different order's data.

This journey requires reading a value on one page, navigating away, and comparing it against a value shown elsewhere — there's no single element that tells you the check passed, so your automation needs to carry state between pages/steps and assert equality across them.

You do not need to touch any other part of the app — this journey is your complete scope.

Requirements

- **BDD is required.** Write your scenarios in Given/When/Then form (e.g. Cucumber, SpecFlow, Behave, or an equivalent for your language/tool of choice) before or alongside the automation code.
- **Tool choice is free,** but Selenium, Playwright, or Cypress are preferred.
- **Test reporting is required.** Your suite must produce a report artifact (HTML report, Allure report, or similar) that a reviewer can open after a run to see which scenarios passed or failed.
- **The suite must run on any machine.** Provide explicit setup and run instructions (e.g. a [README](#)) that let a reviewer clone your repository, install dependencies, and run the suite without editing any file paths, URLs, or credentials in your code. Any environment-specific value (base URL, login credentials) must be supplied via a config file, environment variable, or command-line argument documented in your instructions — never hardcoded to one machine's file system.

The DevTools Network tab is fair game

The app makes real API calls. You're welcome to open your browser's developer tools and inspect the **Network** tab (and the **Application** / storage tabs, and the **Console**) while you build your automation, if that helps you understand the app's behavior.

Test data

Your reviewer will send you a **personal access link** (it looks like <https://.../start?code=...>). Open it once — in the browser your suite drives — and **before you sign in**: it unlocks the app for a limited window, and it is what ties your session to this assignment. If your suite runs in a fresh browser profile each time, make opening that link the first step of your setup, ahead of the login step.

The link is issued for this automation assignment, so the account below is the only one it will sign in with — any other account is rejected.

Sign in with this account. It comes with a saved address, insurance details, and order history so the app has content to work with.

Email	Password
steve@example.test	steve1234

There is no self-registration for this assessment — always use this account.

Submit your automation project (source code, BDD feature files, and a generated test report from a run) along with a short note on any assumptions you made or issues you noticed along the way. The clarity and structure of your project is part of what we assess, alongside whether the suite runs successfully on our end.